StateOfStructure

Documentation

Alexa Katharina-Ronja Brockmöller

June 8, 2026

Contents

1 Introduction

StateOfStructure is an educational desktop tool for students in computer science courses. Its purpose is to make the changing state of data structures visible while Java code is executed. Many students can read an operation such as `push`, `add`, `remove`, or `put`, but still struggle to imagine the resulting internal structure. StateOfStructure addresses this problem by recording intermediate states and presenting them as navigable visual steps.

The tool is useful for programming courses, data-structure courses, algorithm courses, and practical tutorials. It helps students connect source code with concrete state changes in lists, stacks, sets, maps, trees, and course-specific data structures.

2 Learning Goals

StateOfStructure supports the following learning goals:

- understanding how individual Java statements change a data structure;
- observing data structures step by step instead of only seeing a final result;
- comparing a logical data-structure visualization with an object-oriented memory visualization;
- improving debugging intuition by connecting code lines to runtime state;
- experimenting with small Java examples in a safe, focused environment.

3 Prerequisites

Before using StateOfStructure, install the following software:

1. A Java Development Kit (JDK). The Maven configuration currently targets Java release `25`.
2. Maven, or the Windows Maven Wrapper included in the repository as `mvnw.cmd`.
3. Graphviz. Graphviz is required for rendering graph images. It can be downloaded from the official Graphviz download page.

After installing Graphviz, ensure that the Graphviz executables are available on the system path. If Graphviz is not available, graph rendering may fail even if the Java project compiles successfully.

4 Starting the Application

On Windows, run:

```
.\mvnw.cmd clean compile exec:exec
```

On macOS or Linux, run:

```
mvn clean compile exec:exec
```

StateOfStructure is a Swing desktop application. Therefore, it needs a graphical desktop environment. It is not intended to be used only inside a headless terminal session.

5 Basic Workflow

The main workflow is:

1. Start the application.
2. Edit the Java example in the code panel.
3. Create a supported data structure.
4. Register the data structure with `Visualizer.register(...)`.
5. Add operations that mutate the structure.
6. Click **Compile**.
7. Use **Previous** and **Next** to inspect the recorded states.

The tool compiles the user code, records states, and updates the interface with a sequence of visualization steps.

6 Example Program

A minimal example is shown below.

```
import java.util.Stack;
import com.hhu.util.Visualizer;

public class StateOfStructure {

    public void main() {
        Stack<Integer> stack = new Stack<>();
        Visualizer.register(stack);

        stack.push(1);
        stack.push(2);
        stack.pop();
    }
}
```

This example creates a stack, registers it for visualization, pushes two elements, and removes one element. StateOfStructure records the changes and allows the student to move through the resulting states.

7 Rules for User Programs

User programs must follow these conventions so that the tool can compile and execute them correctly:

- The class must be named `StateOfStructure`.
- The method signature must be `public void main()`.

- The code must call `Visualizer.register(dataStructure)` before the operations that should be recorded.
- The registered object should be one of the supported data structures.

8 Supported Data Structures

Data structure
`ArrayList`
`LinkedList`
`Stack`
`HashSet`
`LinkedHashSet`
`TreeSet`
`TreeMap`
`PrograLinkedList`
`PrograHashSet`
`PrograBinarySearchTree`

9 Interface Overview

The graphical interface contains several panels and controls:

Code panel Shows the Java example and highlights the relevant code context for the current step.

Data-structure panel Shows a logical visualization of the registered data structure.

Memory panel Shows an object/memory-oriented view of the registered data structure.

Compile button Compiles the current source code and rebuilds the visualization sequence.

Previous button Moves to the previous recorded state.

Next button Moves to the next recorded state.

Help button Opens an in-application tutorial.

Graph panels support zooming with the mouse wheel, dragging for panning, and double-clicking for zoom toggling.

10 Internal Architecture

StateOfStructure is organized into four main areas.

10.1 Visualization API

The `Visualizer` class is the simple API used by student examples. It stores the active recording object and delegates state recording to the internal draw-call logic. A user program only needs to call `Visualizer.register(...)`. The application then records the states needed for visualization.

10.2 Compilation Pipeline

The compiler component writes the edited user code into a temporary workspace and compiles it with the Java compiler from the JDK. It then loads the compiled `StateOfStructure` class and invokes its non-static `main` method. To make step-by-step visualization possible, the compiler augments the user code by adding recording calls after relevant statements in the `main` method. This allows students to write normal-looking data-structure operations while the tool captures the intermediate states automatically.

10.3 How State Recording Works

At startup, the program reads the default Java snippet from the resources directory. The internal compiler component then prepares the code for execution. It searches for the user method `public void main()` and for the first call to `Visualizer.register(...)`.

After the registered data structure is found, the tool automatically inserts recording calls after later lines in the method body. Conceptually, the student writes this:

```
stack.push(1);
stack.push(2);
stack.pop();
```

The tool treats it like this during execution:

```
stack.push(1); Visualizer.record();
stack.push(2); Visualizer.record();
stack.pop(); Visualizer.record();
```

Each call to `Visualizer.record()` creates a new step in the visual timeline. The timeline can then be explored through the navigation controls in the user interface.

10.4 Graph Construction

The graph builder layer maps supported Java and course-specific data structures to specialized graph builders. These builders create graph descriptions, which are rendered through Graphviz. This separation makes it possible to add more data structures later without rewriting the user interface.

10.5 User Interface

The user interface is implemented with Java Swing. It displays the current recorded step and provides navigation controls. Each recorded step contains the panels needed to show code, memory, and data-structure information together.

11 Testing

The test suite can be executed with:

```
mvn clean install
```

On Windows, use:

```
.\mvnw.cmd clean install
```

The tests cover utility logic, draw-call behavior, UI components, and graph builders for supported data structures.

12 Troubleshooting

12.1 Graphviz Problems

If visualizations do not render, first verify that Graphviz is installed and available on the command line. Restart the terminal or IDE after changing the system path.

12.2 No Java Compiler Found

If the application reports that no Java compiler was found, the program is probably running with a JRE instead of a JDK. Install a JDK and ensure that the application uses it.

13 Extending the Tool

To add support for another data structure, the project should be extended in the graph builder layer. A new builder should translate the structure into a graph representation, and the central graph dispatch logic should be updated to call the new builder for the new type.

14 Conclusion

StateOfStructure helps students understand data structures by connecting Java code to visual state changes. It encourages experimentation, supports step-by-step reasoning, and provides an approachable bridge between abstract theory and concrete runtime behavior.